

江南大学

人工智能与计算机学院

《算法分析与设计》实验报告

专业名称： 计算机科学与技术

课程名称： 算法分析与设计

实验题目： 二维最接近点对问题

实验环境： Windows, C++17, Python 3

姓 名： 丁嘉年

学 号： 1030424316

指导教师： 晏涛

提交日期： 2026 年 4 月 27 日

目录	1
目录	
一、 实验目的	2
二、 实验内容与要求	2
三、 问题分析	2
四、 算法设计	3
4.1 距离计算	3
4.2 整体算法步骤	3
4.3 分治划分与条带区域示意	4
4.4 条带区域按纵坐标排序的原因	4
4.5 递推公式	5
五、 算法正确性与最优性分析	5
5.1 正确性分析	5
5.2 条带区域检查点数的证明	6
5.3 最优性分析	7
六、 时间复杂度分析	7
6.1 分治算法复杂度	7
6.2 与暴力算法比较	8
七、 实验结果	8
7.1 样例测试	8
7.2 随机测试数据说明	8
7.3 不同规模下的运行时间测试	9
7.4 时间曲线分析	10
八、 关键代码分析	11
8.1 点结构体	11
8.2 距离计算函数	11
8.3 小规模点集的暴力处理	12
8.4 分治求解函数	12
九、 实验总结	13
A 附录：相关代码	14

一、 实验目的

本实验围绕二维最接近点对问题展开，主要目标如下：

1. 理解二维平面中最接近点对问题的定义和求解目标；
2. 掌握利用分治算法求解最近点对问题的基本思想；
3. 分析分治算法的递推公式、正确性、最优性和时间复杂度；
4. 将分治算法与暴力枚举算法进行比较，观察二者随输入规模增大时运行时间的变化趋势；
5. 通过图表方式展示算法运行结果，加深对算法效率差异的理解。

二、 实验内容与要求

给定平面上的 n 个点，每个点由横坐标 x 和纵坐标 y 表示。实验要求找出所有点对中距离最小的一对点，并输出该最短距离，结果四舍五入保留两位小数。

输入形式如下：

- 第一行输入整数 n ，表示点的个数；
- 接下来 n 行，每行输入两个整数 x, y ，表示一个点的坐标。

其中，数据范围为：

$$1 \leq n \leq 10^5, \quad 0 \leq x, y \leq 10^9$$

输出形式为一行实数，表示所有点对中的最短距离，保留两位小数。

三、 问题分析

二维最接近点对问题的直接做法是枚举所有点对，计算每一对点之间的欧几里得距离，然后取最小值。若点数为 n ，则点对数量为：

$$C_n^2 = \frac{n(n-1)}{2}$$

因此暴力算法的时间复杂度为 $O(n^2)$ 。当 n 较小时，这种方法容易实现，也能得到正确答案；但本题中 n 最大可达到 10^5 ，此时点对数量约为 5×10^9 ，暴力算法运行时间过长，难以满足要求。

因此，本实验采用分治算法。分治算法的核心思想是：先按照横坐标将所有点排序，然后把点集划分成左右两部分，分别求出左右两边内部的最接近点对距离，再处理

可能跨越分界线的最近点对。通过这种方式，可以将原本的二重枚举优化为递归求解和线性合并。

需要注意的是，仅仅递归求解左右两个子问题是不够的。因为全局最近点对可能由一个左侧点和一个右侧点组成。如果只取左右两侧内部最近距离的最小值，就会遗漏跨越分界线的情况。因此，分治算法的关键不仅在于划分子问题，还在于合并阶段对分界线附近条带区域的处理。

四、 算法设计

4.1 距离计算

对于两个点 $P_i(x_i, y_i)$ 和 $P_j(x_j, y_j)$ ，它们之间的欧几里得距离为：

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

程序中使用 `double` 类型存储距离，避免整数运算无法表示小数距离的问题。

4.2 整体算法步骤

本实验采用分治算法求解二维最近点对问题，主要步骤如下：

1. 读入所有点，并为每个点设置编号 `id`，用于在递归划分时区分点的归属；
2. 将点集分别按照 x 坐标和 y 坐标排序，得到数组 `px` 和 `py`；
3. 若当前点集规模不超过 3，直接使用暴力枚举计算最近距离；
4. 否则，按照 x 坐标的中位位置将点集划分为左半部分和右半部分；
5. 递归求解左半部分最近距离 d_L 和右半部分最近距离 d_R ；
6. 令当前最短距离 $d = \min(d_L, d_R)$ ；
7. 构造条带区域，即所有满足 $|x - midX| < d$ 的点；
8. 条带区域中的点已经按照 y 坐标有序，因此对每个点只需检查其后面有限个点；
9. 更新最短距离，返回最终答案。

4.3 分治划分与条带区域示意

在分治算法中，首先将所有点按照横坐标排序，然后取中间位置作为划分依据，将点集划分为左、右两个子问题，分别递归求解左右两部分内部的最近点对距离。设左右两部分求得的最近距离分别为 d_L 和 d_R ，令当前最短距离为

$$\delta = \min(d_L, d_R)$$

此时，除了左右子问题内部的最近点对外，还需要进一步检查是否存在一对最近点分别位于分界线两侧。由于只有距离分界线小于 δ 的点才可能构成更优解，因此合并阶段只需要检查分界线附近宽度为 2δ 的条带区域。

图 1 展示了分治算法中的左右划分以及条带区域的基本形式。

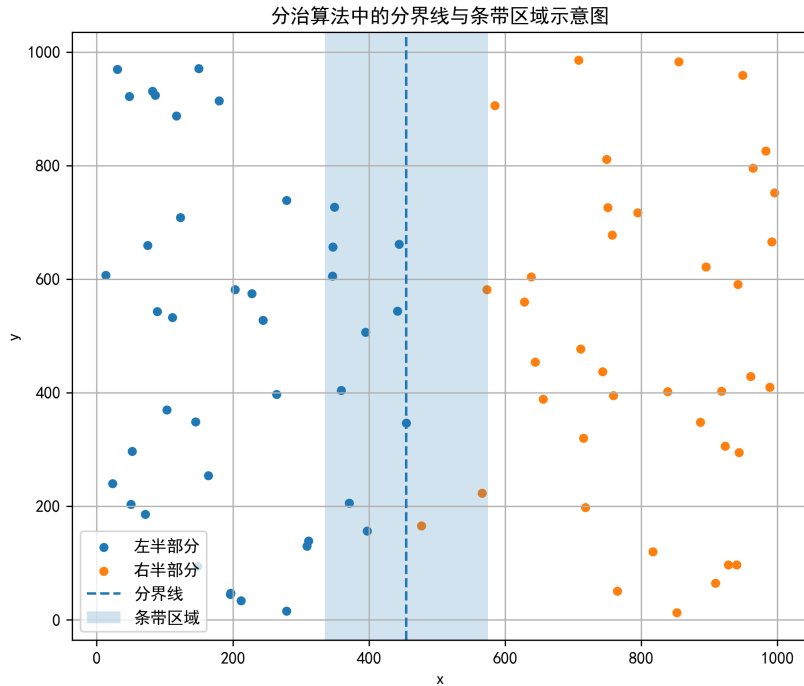


图 1: 分治算法中的分界线与条带区域示意图

如图 1 所示，分界线将点集分成左右两部分。递归阶段分别求解左右子问题，合并阶段则只需检查中间条带中的候选点，而不需要比较所有跨区域点对，从而有效降低计算量。

4.4 条带区域按纵坐标排序的原因

在分治算法的合并阶段，需要检查可能跨越分界线的最近点对。若直接对条带区域中的所有点两两比较，仍可能产生较高的时间开销。因此，程序中保持条带区域中的点按照 y 坐标升序排列。

这样做的好处是：对于条带中的某个点 $P_i(x_i, y_i)$ ，只需要继续检查其后面纵坐标差小于当前最短距离 δ 的点。如果某个后续点 $P_j(x_j, y_j)$ 已经满足

$$y_j - y_i \geq \delta$$

那么两点之间的欧几里得距离一定满足

$$d(P_i, P_j) \geq |y_j - y_i| \geq \delta$$

因此，该点以及其后面纵坐标更大的点都不可能使当前最短距离变小，可以直接停止检查。

所以，按照 y 坐标排序能够让条带区域的检查从无序比较变成顺序扫描，并配合“最多检查常数个候选点”的性质，使合并阶段保持 $O(n)$ 的时间复杂度。

4.5 递推公式

设 $T(n)$ 表示处理 n 个点所需的时间。每次递归时，算法将点集划分为两个规模约为 $n/2$ 的子问题。合并阶段需要扫描按 y 排序的点集，并检查中间条带区域，时间为 $O(n)$ 。

因此递归部分满足：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

由主定理可得：

$$T(n) = O(n \log n)$$

初始阶段需要对点集按 x 坐标和 y 坐标进行排序，排序时间复杂度为 $O(n \log n)$ 。因此，整个分治算法的总时间复杂度仍为：

$$O(n \log n)$$

五、 算法正确性与最优性分析

5.1 正确性分析

对于任意一个点集，最近点对只可能出现在以下三种情况中：

1. 最近点对的两个点都在左半部分；
2. 最近点对的两个点都在右半部分；

3. 最近点对的两个点分别位于分界线两侧。

分治算法递归求解左半部分和右半部分，因此能够分别得到左半部分内部和右半部分内部的最短距离。设这两个距离分别为 d_L 和 d_R ，则当前已知最短距离为：

$$d = \min(d_L, d_R)$$

接下来只需要考虑跨越分界线的点对。如果某个跨越分界线的点对距离小于 d ，那么这两个点到分界线的横向距离都必须小于 d 。因此，它们一定出现在宽度为 $2d$ 的中间条带区域中。

程序将条带区域中的点按照 y 坐标顺序处理。对于条带中的某个点，只需要检查另一侧最多 6 个相关点。

综上，算法分别考虑了左半部分、右半部分和跨分界线三种情况，没有遗漏任何可能的最近点对，因此算法是正确的。

5.2 条带区域检查点数的证明

分治算法递归求出左右两部分的最近距离后，设

$$\delta = \min(d_L, d_R)$$

此时，左半部分内部任意两点之间的距离都不小于 δ ，右半部分内部任意两点之间的距离也不小于 δ 。因此，如果还存在距离小于 δ 的点对，那么这两个点一定分别位于分界线两侧。

设分界线为 l ，取左侧的任意一点 $P_i(x_i, y_i)$ 。若右侧某点 P_j 与 P_i 的距离小于 δ ，则 P_j 的纵坐标必须满足

$$y_i - \delta \leq y_j \leq y_i + \delta$$

并且它必须位于分界线右侧宽度不超过 δ 的区域内。所以，对固定点 P_i 来说，真正需要检查的右侧候选点只可能位于一个宽为 δ 、高为 2δ 的矩形中。

将该矩形划分为 6 个相等的小矩形，每个小矩形的宽为 $\delta/2$ ，高为 $2\delta/3$ 。此时每个小矩形的对角线长度为

$$d = \sqrt{\left(\frac{\delta}{2}\right)^2 + \left(\frac{2\delta}{3}\right)^2} = \delta \sqrt{\frac{1}{4} + \frac{4}{9}} = \frac{5\delta}{6} < \delta$$

也就是说，同一个小矩形中任意两点之间的距离一定小于 δ 。但是右侧子问题内部的最近距离已经不小于 δ ，所以每个小矩形中最多只能有一个右侧点。

由于一共有 6 个小矩形，因此对于固定的左侧点 P_i ，右侧可能与它构成距离小于 δ 的候选点最多只有 6 个。同理，固定右侧点时，左侧需要检查的候选点也最多只有 6 个。

所以，在条带区域中按纵坐标顺序检查时，每个点只需要检查常数个候选点。这样，合并阶段的总时间为 $O(n)$ ，不会退化为暴力枚举。

5.3 最优性分析

算法在每一层递归中都会返回当前点集内所有可能点对距离的最小值。对于规模较小的基本情况，算法直接枚举所有点对，能够得到最优解。

对于规模较大的情况，算法将问题划分为左半部分、右半部分和跨分界线三类。左半部分和右半部分的最优解由递归保证；跨分界线的可能最优解由条带检查保证。最终算法取三类情况中的最小值，因此返回的一定是当前点集的全局最短距离。

所以，该分治算法能够得到二维最近点对问题的最优解。

六、 时间复杂度分析

6.1 分治算法复杂度

本实验算法主要包含排序、递归划分和条带合并三个部分。

1. 排序：对所有点分别按照 x 坐标和 y 坐标排序，平均时间复杂度为 $O(n \log n)$ ；
2. 递归划分：每次将点集划分为两个规模约为 $n/2$ 的子问题；
3. 合并检查：每一层递归中，构造左右子集和条带区域的总扫描时间为 $O(n)$ 。

因此递归部分的时间复杂度满足：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

由主定理可得：

$$T(n) = O(n \log n)$$

加上初始排序后，总时间复杂度仍为：

$$O(n \log n)$$

空间复杂度方面，递归过程中需要存储左右子数组、条带数组和辅助集合，因此空间复杂度约为：

$$O(n \log n)$$

6.2 与暴力算法比较

暴力算法需要枚举所有点对，时间复杂度为：

$$O(n^2)$$

分治算法的时间复杂度为：

$$O(n \log n)$$

当 n 较小时，暴力算法实现简单，运行时间也可以接受；但当 n 增大时， $O(n^2)$ 的增长速度明显快于 $O(n \log n)$ 。因此，对于本题 $n \leq 10^5$ 的数据范围，分治算法更适合。

七、 实验结果

7.1 样例测试

输入数据为：

```
1 4
2 0 0
3 0 1
4 1 0
5 1 1
```

程序输出为：

```
1 1.00
```

四个点构成一个边长为 1 的正方形，最近点对距离为 1，程序输出结果正确。

7.2 随机测试数据说明

为了测试算法在不同输入规模下的运行效果，本实验随机生成二维点集作为测试数据。点的横坐标和纵坐标均在指定范围内随机产生，不同规模的数据分别用于测试暴力算法和分治算法的运行时间。

图 2 展示了随机生成的二维点集分布情况。从图中可以看出，测试点在平面区域内随机分布，没有明显的特殊排列规律。因此，该数据可以用于观察算法在一般随机输入下的运行表现。

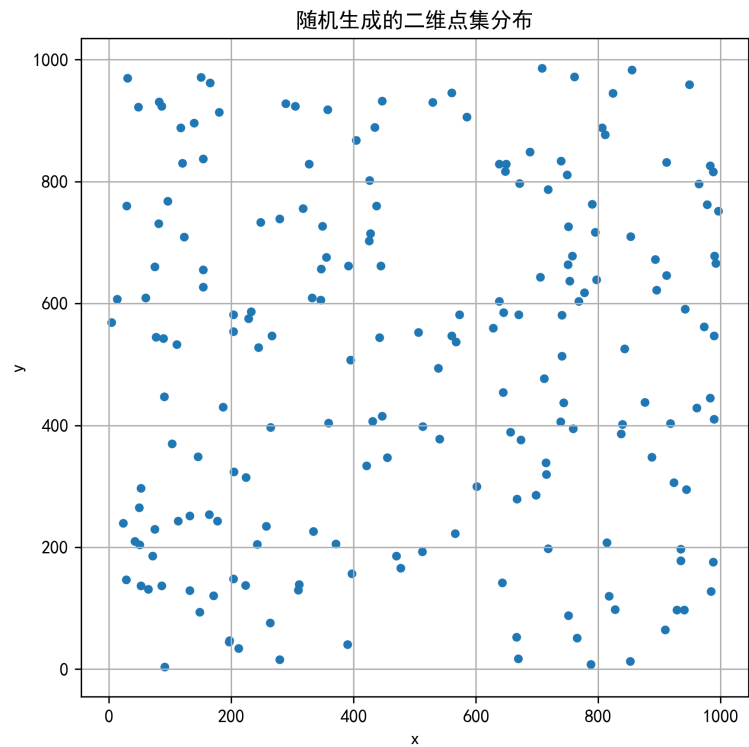


图 2: 随机生成的二维点集分布

7.3 不同规模下的运行时间测试

为了观察算法随输入规模增大时的运行趋势，本实验额外随机生成不同规模的二维点集，并分别记录暴力算法和分治算法的运行时间。测试时，每个规模运行多次并取平均值，以减少偶然误差。两种算法使用相同的输入数据，以保证实验对比的公平性。

表 1: 不同输入规模下的运行时间对比

点数 n	暴力算法运行时间 / ms	分治算法运行时间 / ms
100	0.15548	0
500	3.03376	0.41696
1000	8.20876	1.74168
2000	37.40930	4.29378
4000	157.96100	9.11268
8000	602.18300	17.11160
16000	2383.64000	38.88030
32000	11462.40000	99.62170

从表中可以看出，随着输入规模增大，暴力算法运行时间增长较快，而分治算法运行时间增长较慢。当输入规模较小时，分治算法运行时间可能接近计时器精度下限，

因此个别小规模测试结果可能显示为 0，但这不影响对整体增长趋势的分析。

7.4 时间曲线分析

图 3 展示了普通坐标下暴力算法与分治算法的运行时间对比。从图中可以看出，随着输入规模增大，暴力算法运行时间增长很快，而分治算法的增长速度明显更慢。这与理论分析中暴力算法 $O(n^2)$ 、分治算法 $O(n \log n)$ 的结论一致。

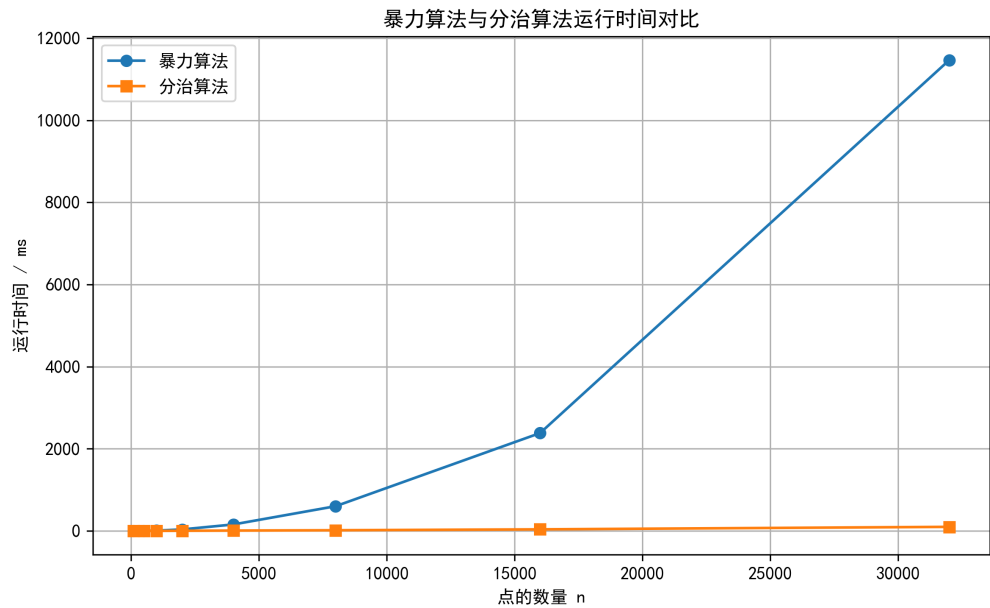


图 3: 暴力算法与分治算法运行时间对比

由于暴力算法和分治算法的运行时间差距较大，普通坐标下分治算法曲线显得较为平缓。为了更清楚地观察两种算法的增长趋势，本实验进一步使用对数坐标绘制运行时间曲线，如图 4 所示。

从对数坐标图中可以看出，暴力算法的运行时间仍然随输入规模快速上升，而分治算法增长更加平缓。这进一步说明，分治算法在较大规模输入下具有更明显的效率优势。

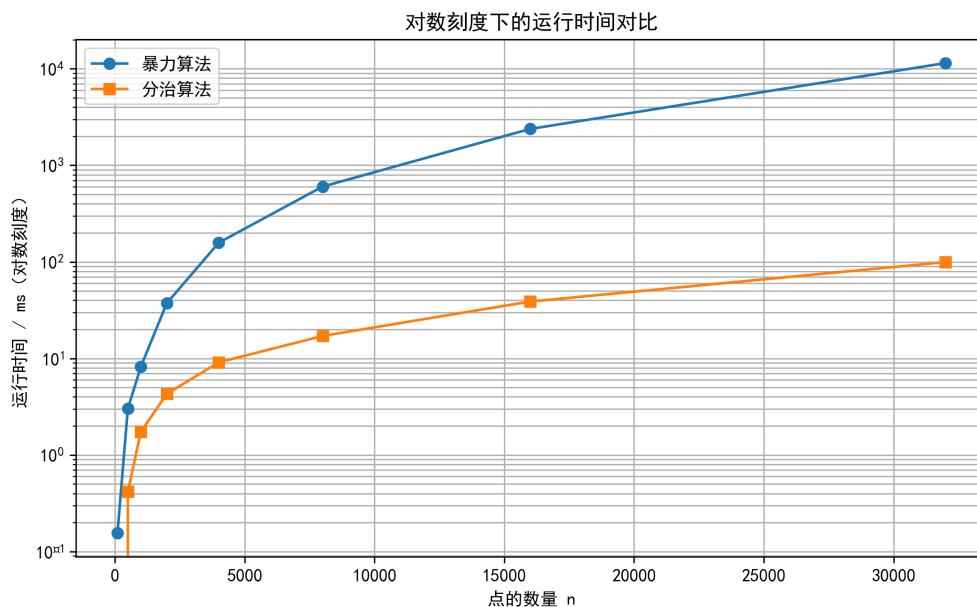


图 4: 对数坐标下的运行时间对比

八、 关键代码分析

8.1 点结构体

程序使用结构体 `Point` 存储点的横坐标、纵坐标和编号。编号 `id` 的作用是在递归划分时判断某个点属于左半部分还是右半部分。

```
1 struct Point
2 {
3     ll x, y;
4     ll id;
5 };
```

8.2 距离计算函数

函数 `dist` 用于计算两个点之间的欧几里得距离。

```
1 double dist(const Point& x, const Point& y)
2 {
3     double dx = x.x - y.x;
4     double dy = x.y - y.y;
5     return sqrt(dx * dx + dy * dy);
6 }
```

8.3 小规模点集的暴力处理

当点数不超过 3 时，直接枚举所有点对。这样既可以作为递归的终止条件，也可以避免在很小规模问题上继续递归带来的额外开销。

```
1 double bruteForce(const vector<Point>& pp)
2 {
3     if (pp.size() == 0 || pp.size() == 1)
4         return 1e100;
5     if (pp.size() == 2)
6         return dist(pp[0], pp[1]);
7     if (pp.size() == 3)
8         return min({dist(pp[0], pp[1]), dist(pp[0], pp[2]), dist(pp[1], pp[2])
9                     });
10    return 1e100;
11 }
```

8.4 分治求解函数

函数 solve 是本实验代码的核心。它先递归求解左右两部分，再检查中间条带区域中可能跨越分界线的最近点对。

```
1 double solve(const vector<Point>& px, const vector<Point>& py)
2 {
3     if (px.size() <= 3)
4         return bruteForce(px);
5
6     ll mid = px.size()/2;
7     double midX = px[mid].x;
8
9     vector<Point> l_px(px.begin(), px.begin() + mid);
10    vector<Point> r_px(px.begin() + mid, px.end());
11    vector<Point> l_py, r_py;
12
13    unordered_set<ll> l_ids;
14    for (int i = 0; i < mid; i++)
15        l_ids.insert(px[i].id);
16
17    for (auto & p : py)
18        if (l_ids.count(p.id))
19            l_py.push_back(p);
```

```
20     else
21         r_py.push_back(p);
22
23     double ans = min(solve(l_px, l_py), solve(r_px, r_py));
24
25     vector<Point> strip;
26     for (auto& p : py)
27         if (fabs(midX - p.x) < ans)
28             strip.push_back(p);
29
30     for (int i = 0; i < strip.size(); i++)
31         for (int j = i + 1; j < strip.size() && j <= i + 6; j++)
32         {
33             if (strip[j].y - strip[i].y >= ans)
34                 break;
35             ans = min(ans, dist(strip[i], strip[j]));
36         }
37
38     return ans;
39 }
```

其中，strip 保存距离中线小于当前最短距离的点。由于 py 本身按照 y 坐标有序，所以 strip 也是按 y 坐标有序的。检查时如果两个点的纵坐标差已经大于等于当前答案，就可以提前结束。

九、 实验总结

通过本实验，我进一步理解了分治算法在几何问题中的应用。二维最近点对问题如果直接使用暴力枚举，需要检查所有点对，时间复杂度为 $O(n^2)$ 。当输入规模较小时，暴力算法实现简单，也容易验证结果；但当点数增大后，点对数量会迅速增加，运行时间也会明显上升。

分治算法通过先排序、再递归划分点集，将原问题拆分为左右两个规模较小的子问题，并在合并阶段只检查分界线附近的条带区域。这样可以避免对所有点对进行枚举。特别是条带区域中“每个点只需要检查常数个候选点”的结论，是分治算法能够达到 $O(n \log n)$ 时间复杂度的关键。

在实验过程中，我不仅验证了算法在样例输入下的正确性，也通过随机生成不同规模的二维点集，对暴力算法和分治算法的运行时间进行了对比。从实验结果和运行时间曲线可以看出，暴力算法的运行时间随输入规模增大而快速增长，而分治算法的增长趋势更加平缓，这与理论复杂度分析基本一致。

本实验也让我认识到，算法实现时不仅要理解核心思想，还要注意一些细节。例如，条带区域需要按照纵坐标顺序检查，递归终止条件需要正确设置，距离计算需要使用浮点类型。此外，运行时间测试还需要注意计时精度和重复实验次数，否则在小规模输入下可能出现计时结果不明显的问题。

总体来说，本实验加深了我对分治思想的理解。分治算法不是简单地把问题拆开求解，还需要设计有效的合并过程。对于二维最近点对问题，正是由于合并阶段能够被限制在线性时间内，整个算法才比暴力枚举具有更好的效率。

A 附录：相关代码

本实验的完整代码及相关分析脚本已上传至 GitHub 仓库，链接如下：<https://github.com/anyuan-git-hub/-/tree/main/%E4%BA%8C%E7%BB%B4%E6%9C%80%E6%8E%A5%E8%BF%91%E7%82%B9%E5%AF%B9%E9%97%AE%E9%A2%98>

该仓库主要包括二维最近点对问题的 C++ 实验程序、运行时间测试脚本、随机数生成脚本以及实验结果可视化代码。下面给出本实验所使用的核心 C++ 程序代码。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct Point
6 {
7     ll x, y;
8     ll id;
9 };
10
11 bool cmp_x(const Point& x, const Point& y)
12 {
13     if (x.x != y.x)
14         return x.x < y.x;
15     return x.y < y.y;
16 }
17
18 bool cmp_y(const Point& a, const Point& b)
19 {
20     if (a.y != b.y)
21         return a.y < b.y;
22     return a.x < b.x;
23 }
```

```
24
25 void quickSort(vector<Point> &p, ll l, ll r, bool (*cmp)(const Point&, const
    Point&))
26 {
27     if (l >= r)
28         return;
29
30     ll i = l, j = r;
31     Point pivot = p[(l+r)/2];
32
33     while (i <= j)
34     {
35         while (cmp(p[i], pivot)) i++;
36         while (cmp(pivot, p[j])) j--;
37
38         if (i <= j)
39         {
40             swap(p[i], p[j]);
41             i++;
42             j--;
43         }
44     }
45
46     quickSort(p, l, j, cmp);
47     quickSort(p, i, r, cmp);
48 }
49
50 double dist(const Point& x, const Point& y)
51 {
52     double dx = x.x - y.x;
53     double dy = x.y - y.y;
54     return sqrt(dx * dx + dy * dy);
55 }
56
57 double bruteForce(const vector<Point>& pp)
58 {
59     if (pp.size() == 0 || pp.size() == 1)
60         return 1e100;
61     if (pp.size() == 2)
62         return dist(pp[0], pp[1]);
```



```

63     if (pp.size() == 3)
64         return min({dist(pp[0], pp[1]), dist(pp[0], pp[2]), dist(pp[1], pp[2])
65                     });
66     return 1e100;
67 }
68 double solve(const vector<Point>& px, const vector<Point>& py)
69 {
70     if (px.size() <= 3)
71         return bruteForce(px);
72
73     ll mid = px.size()/2;
74     double midX = px[mid].x;
75
76     vector<Point> l_px(px.begin(), px.begin() + mid);
77     vector<Point> r_px(px.begin() + mid, px.end());
78     vector<Point> l_py, r_py;
79
80     unordered_set<ll> l_ids;
81     for (int i = 0; i < mid; i++)
82         l_ids.insert(px[i].id);
83
84     for (auto & p : py)
85         if (l_ids.count(p.id))
86             l_py.push_back(p);
87         else
88             r_py.push_back(p);
89
90     double ans = min(solve(l_px, l_py), solve(r_px, r_py));
91
92     vector<Point> strip;
93     for (auto& p : py)
94         if (fabs(midX - p.x) < ans)
95             strip.push_back(p);
96
97     for (int i = 0; i < strip.size(); i++)
98         for (int j = i + 1; j < strip.size() && j <= i + 6; j++)
99             {
100                 if (strip[j].y - strip[i].y >= ans)
101                     break;

```

```
102         ans = min(ans, dist(strip[i], strip[j]));
103     }
104
105     return ans;
106 }
107
108 int main()
109 {
110     ll n;
111     cin >> n;
112
113     vector<Point> a(n), px(n), py(n);
114     for (int i = 0; i < n; i++)
115     {
116         cin >> a[i].x >> a[i].y;
117         a[i].id = i;
118         px[i] = a[i];
119         py[i] = a[i];
120     }
121
122     if (n < 2)
123     {
124         cout << fixed << setprecision(2) << 0.00 << endl;
125         return 0;
126     }
127
128     quickSort(px, 0, n - 1, cmp_x);
129     quickSort(py, 0, n - 1, cmp_y);
130
131     double ans = solve(px, py);
132
133     cout << fixed << setprecision(2) << ans << endl;
134
135     return 0;
136 }
```